



ESCUELA POLITÉCNICA NACIONAL

PROYECTO – PROGRAMACIÓN I 2026-A

SISTEMA DE VOTACIONES AEIS

Grupo: 3

Integrantes: Anderson Verdezoto, Erick Cuji, Jairo Enríquez.

Curso: GR2CC

Docente: Ing. Tomás Borja, MSc.

Fecha: miércoles 15 de julio de 2026.

1. Objetivos

1.1 Objetivo General

Desarrollar e implementar un software modular en lenguaje C que automatice el proceso de votaciones para la directiva de la AEIS, garantizando la integridad de los datos, el control del padrón electoral y la jerarquización automatizada de los resultados.

1.2 Objetivos específicos

Garantizar el principio de "un estudiante, un voto" mediante la validación del código único contra el padrón indexado utilizando búsqueda binaria.

Asegurar la persistencia de datos almacenando la configuración, candidatos y votos en archivos de texto plano (.txt) legibles y duraderos.

Automatizar el desglose estadístico final calculando los porcentajes exactos de participación y ausentismo de forma inmediata al cerrar las votaciones.

2. Proceso de diseño lógico

BLOQUE 1: Módulo Principal (main)

Controla el flujo macro del sistema mediante un bucle interactivo que divide el acceso para los dos roles de usuario (Estudiantes y Administrador).

```
Algoritmo Sistema_Votaciones_AEIS
  Definir opcion Como Entero
  opcion <- 0

  Mientras opcion != 3 Hacer
    Mostrar "===== "
    Mostrar "      ELECCIONES PARA LA PRESIDENCIA DE LA AEIS  "
    Mostrar "===== "
    Mostrar "1. Panel de Estudiantes"
    Mostrar "2. Panel de Administrador"
    Mostrar "3. Salir"
    Mostrar "Opcion: "
    Leer opcion

    Segun opcion Hacer
      1: Enrutar a Modulo_Panel_Estudiantes()
      2: Enrutar a Modulo_Panel_Administrador()
      3: Mostrar "Saliendo del Programa de Votaciones de la AEIS."
      De Otro Modo: Mostrar "Opcion invalida."
    FinSegun
  FinMientras
FinAlgoritmo
```

BLOQUE 2: Control de Autenticación (panelAdministrador)

Este bloque gestiona la validación de acceso administrativo mediante credenciales del sistema, desplegando el submenú de gestión.

```
Subproceso PanelAdministrador()  
  Definir usuario, contrasena Como Cadena  
  Definir opcion_admin Como Entero  
  
  Escribir "Ingrese usuario: "  
  Leer usuario  
  Escribir "Ingrese contrasena: "  
  Leer contrasena  
  
  Si usuario = "POLIADMIN" Y contrasena = "admin123" Entonces  
    Escribir "SESION INICIADA EXITOSAMENTE"  
    opcion_admin <- 0  
    Mientras opcion_admin != 6 Hacer  
      Escribir ""  
      Escribir "--- MENU ADMINISTRADOR ---"  
      Escribir "1. Nuevas Votaciones de AEIS"  
      Escribir "2. Continuar Votaciones Previas"  
      Escribir "3. Ver Resultados y Exportar Acta (.txt)"  
      Escribir "4. Anadir Candidato/Lista"  
      Escribir "5. Quitar Candidato/Lista"  
      Escribir "6. Cerrar Sesion"  
      Escribir "Opcion: "  
      Leer opcion_admin  
  
      Segun opcion_admin Hacer  
        1: Llamar NuevaEleccion()  
        2: Escribir "Proceso previo restaurado con éxito."  
        3: Llamar MostrarResultados()  
        4: Llamar AgregarCandidatoDinamico()  
        5: Llamar QuitarCandidatoDinamico()  
        6: Escribir "Cerrando sesion de administrador..."  
      FinSegun  
    FinMientras  
  Sino  
    Escribir "Usuario o Contrasena Incorrectos. Acceso Denegado."  
  FinSi  
FinSubproceso
```

BLOQUE 3: Inicialización y Persistencia Inicial (nuevaEleccion)

Establece los parámetros base de la simulación del padrón (20 estudiantes de la EPN) y configura las listas iniciales.

```

Subproceso NuevaEleccion()
  Definir candidatos_reales, i, anio_eleccion Como Entero
  Escribir "Ingrese el Ano actual de las Votaciones (ej: 2026): "
  Leer anio_eleccion

  Si anio_eleccion > 2026 Entonces
    Escribir "[Error] No se puede configurar un proceso posterior al actual."
  Sino
    Escribir "Ingrese el Numero de listas iniciales (Max 10): "
    Leer candidatos_reales

    Para i <- 1 Hasta candidatos_reales Con Paso 1 Hacer
      Escribir "Configurando Lista ", i
      // Lógica para registrar cadenas en variables asignadas
    FinPara

    Escribir "[Padron] Base de datos de 20 votantes generada con exito."
    Escribir "Nuevo proceso electoral configurado de manera limpia."
  FinSi
FinSubproceso

```

BLOQUE 4: Motor de Búsqueda y Validación (studentPanel + Búsqueda Binaria)

Implementa el algoritmo de búsqueda de costo logarítmico para certificar si el votante consta en el padrón electoral.

```

Subproceso PanelEstudiantes()
  Definir id_usuario Como Cadena
  Definir busqueda_idx Como Entero

  Escribir "Ingrese su CODIGO UNICO de estudiante (ej: 202511020): "
  Leer id_usuario

  // Llamada al motor de búsqueda logarítmica O(log n)
  busqueda_idx <- busquedaBinariaVotante(padron_electoral, 20, id_usuario)

  Si busqueda_idx = -1 Entonces
    Escribir "Acceso denegado: El codigo ingresado no consta en el padron oficial de la FIS."
  Sino
    Escribir "Acceso Autorizado. Desplegando papeleta de votacion matricial..."
    // Flujo para procesar seleccion de ID de lista y sumar el voto correspondiente
  FinSi
FinSubproceso

Subproceso indice <- busquedaBinariaVotante(lista, total, id_objetivo)
  Definir izquierda, derecha, medio, indice Como Entero
  izquierda <- 1
  derecha <- total
  indice <- -1

  Mientras izquierda <= derecha Y indice = -1 Hacer
    medio <- Trunc(izquierda + (derecha - izquierda) / 2)
    Si lista[medio] = id_objetivo Entonces
      indice <- medio
    Sino
      Si lista[medio] < id_objetivo Entonces
        izquierda <- medio + 1
      Sino
        derecha <- medio - 1
      FinSi
    FinSi
  FinMientras
FinSubproceso

```

BLOQUE 5: Conteo Recursivo, Ranking y Reportes (mostrarResultados + Selección)

Aplica recursividad para consolidar las actas, jerarquiza de mayor a menor con ordenamiento físico por selección y genera la salida en texto plano.

```
Subproceso MostrarResultados()
  Definir total_votos Como Entero

  // Llamada a función recursiva de conteo
  total_votos <- contarVotosRecursivo(votos_listas, num_candidatos)

  // Llamada al algoritmo de ordenamiento por seleccion para armar el Ranking
  Llamar ordenarCandidatosSeleccion(nombres_listas, votos_listas, num_candidatos)

  Escribir "--- CUADRO DE RESULTADOS FINALES: PRESIDENCIA DE LA AEIS ---"
  Escribir "Total de Votos Escrutados: ", total_votos
  Escribir "[Reportes] Archivo fisico de texto plano 'Acta_Electoral_AEIS.txt' exportado con éxito."
FinSubproceso

Subproceso total <- contarVotosRecursivo(votos_candidatos, n)
  Definir total Como Entero
  Si n <= 0 Entonces
    total <- 0
  Sino
    total <- votos_candidatos[n] + contarVotosRecursivo(votos_candidatos, n - 1)
  FinSi
FinSubproceso

Subproceso ordenarCandidatosSeleccion(nombres, votos, n)
  Definir i, j, max_idx, temp_voto Como Entero
  Definir temp_nombre Como Cadena

  Para i <- 1 Hasta n - 1 Con Paso 1 Hacer
    max_idx <- i
    Para j <- i + 1 Hasta n Con Paso 1 Hacer
      Si votos[j] > votos[max_idx] Entonces
        max_idx <- j
    FinSi
  FinPara
  // Intercambio estructural de posiciones
  temp_voto <- votos[max_idx]
  votos[max_idx] <- votos[i]
  votos[i] <- temp_voto

  temp_nombre <- nombres[max_idx]
  nombres[max_idx] <- nombres[i]
  nombres[i] <- temp_nombre
FinPara
FinSubproceso
```

BLOQUE 6: Segmentación Dinámica de Procesos (agregar y quitar)

Este bloque permite expandir o contraer las dimensiones de la matriz electoral de candidatos sobre la marcha de forma aislada sin comprometer el padrón.

```

Subproceso AgregarCandidatoDinamico()
    Escribir "Ingrese el nombre de la nueva Lista a integrar:"
    // Código para insertar un nuevo binomio desplazando Blanco y Nulo hacia el extremo final
    Escribir "Nueva lista integrada exitosamente al proceso segmentado."
FinSubproceso

Subproceso QuitarCandidatoDinamico()
    Definir id_eliminar Como Entero
    Escribir "Ingrese el ID de la lista que desea retirar de la matriz del proceso: "
    Leer id_eliminar

    Si id_eliminar >= 1 Y id_eliminar <= (num_candidatos - 2) Entonces
        Escribir "La lista seleccionada ha sido dada de baja del proceso."
    Sino
        Escribir "[Error] ID fuera de rango o corresponde a opciones fijas e inamovibles."
    FinSi
FinSubproceso

```

3. Memoria Técnica de Diseño e Implementación: Sistema de Escrutinio AEIS EPN

1. Fundamentación Teórica de la Solución

El diseño de un sistema de escrutinio digital requiere la convergencia de estructuras de datos estáticas y dinámicas, algoritmos estructurados de búsqueda/ordenación y estrategias de persistencia de datos y control de integridad.

1.1 Estructuras de Datos y Modelado de la Entidad Candidato

Para representar cada opción electoral en memoria principal, se definió una estructura Candidato:

- **Campos de identificación y nombres:** id (entero único para referenciar la opción), nombre_lista, presidente y vicepresidente como arreglos de caracteres estáticos de tamaños fijos de 20 y 30 caracteres para evitar desbordamientos de memoria (*buffer overflow*) durante la entrada estructurada de datos.
- **Persistencia de acumulación:** Un campo votos de tipo entero (int) y una bandera de estado binaria activo (int) que determina si una lista se encuentra vigente o ha sido eliminada lógicamente por un administrador.

```

typedef struct {
    int id;
    char nombre_lista[20];
    char presidente[30];
    char vicepresidente[30];
    int votos;
    int activo;
} Candidato;

```

1.2 Algoritmos de Búsqueda y Ordenación Requeridos

- **Búsqueda Binaria de Votantes:** Para garantizar que solo los estudiantes empadronados ejerzan su derecho al voto, se implementó un algoritmo de búsqueda binaria sobre una matriz de cadenas ordenada lexicográficamente.

1.3 Recursividad Aplicada

- **Conteo de Votos de manera Recursiva:** El cálculo acumulado total de votos se realiza a través de un enfoque de "divide y vencerás" con la función contarVotosRecursivo:

2. Planificación y Arquitectura Modular

El proyecto se segmentó en un modelo modular limpio de tres capas lógicas para asegurar la escalabilidad y facilitar el mantenimiento de la aplicación.

```
[ main.c ] <-- Interfaz de Menú Principal (Flujo Principal)
    |
    ▼
[ election.h ] <-- Declaraciones de Tipos, Constantes y Prototipos
    |
    ▼
[ gestion.c ] <-- Capa de Negocio, Archivos, Validación y Algoritmos
```

2.1 Módulo election.h (Definición e Interfaces)

Contiene las directivas de preprocesamiento, la definición de la estructura del candidato y las firmas de todas las funciones operacionales del sistema. Evita la inclusión múltiple mediante directivas `#ifndef ELECTION_H`.

2.2 Módulo gestion.c (Lógica de Negocio y Persistencia)

Concentra las variables globales del estado electoral (`eleccion_activa`, `num_candidatos`, `total_votos_emitidos`), el manejo dinámico de memoria para los registros de votantes (`registro_votos_global` con asignación vía `realloc/malloc`) y la persistencia en texto plano mediante archivos del sistema (`ElectionInfo.txt`, `candidateX.txt`, `Acta_Electoral_AEIS.txt`).

2.3 Módulo main.c (Controlador de Interfaz y Seguridad)

Gestiona la navegación del usuario mediante menús numéricos interactivos y provee un módulo de validación seguro para el administrador (`POLIADMIN / admin123`) con enmascaramiento de contraseñas por caracteres asterisco `*` mediante lectura no-eco por teclado (`_getch` en Windows o comandos `termios` en UNIX).

3. Codificación e Implementación de Procesos Clave

A continuación se analizan los flujos de código más críticos del archivo `gestion.c` que dan soporte a las reglas electorales:

3.1 Gestión Dinámica de Candidatos

El sistema permite la inserción dinámica de nuevas listas a través de `agregarCandidatoDinamico`. Para no interferir con las posiciones obligatorias del **Voto Blanco** y **Voto Nulo** (que deben ir al final), el algoritmo desplaza estas dos opciones fijas una posición a la derecha antes de registrar la nueva propuesta:

```
void agregarCandidatoDinamico() {
    if (num_candidatos >= MAX_CANDIDATOS) {
        printf("[Error] Limite maximo de listas alcanzado.\n");
        return;
    }
    // Desplazamiento físico de las opciones obligatorias Blanco y Nulo
    lista_candidatos[num_candidatos] = lista_candidatos[num_candidatos - 1];
    lista_candidatos[num_candidatos - 1] = lista_candidatos[num_candidatos - 2];

    int idx = num_candidatos - 2; // Índice donde se ubicará la nueva lista
    lista_candidatos[idx].id = idx + 1;
    /* ... Entrada de datos (Nombre de lista, Presidente, Vicepresidente) ... */
    lista_candidatos[idx].votos = 0;
    lista_candidatos[idx].activo = 1;

    lista_candidatos[num_candidatos - 1].id = num_candidatos;
    lista_candidatos[num_candidatos].id = num_candidatos + 1;

    num_candidatos++;
    guardarConfiguracionEleccion();
}
```

3.2 Protección contra el Doble Voto

Durante la ejecución del panel del estudiante, se valida que el código ingresado no conste dentro de la matriz dinámica `registro_votos_global`. Si el votante es aprobado, su identificador se anexa dinámicamente utilizando `realloc`:


```

for (int i = 0; i < total_votos_emitidos; i++) {
    if (strcmp(registro_votos_global[i], id_usuario) == 0) {
        printf("El codigo ingresado YA REGISTRA UN VOTO.\n");
        return;
    }
}
// Expansión dinámica del registro global de votos emitidos
char** temporal = realloc(registro_votos_global, (total_votos_emitidos + 1) * sizeof
if (temporal == NULL) { return; }
registro_votos_global = temporal;
registro_votos_global[total_votos_emitidos] = malloc(15 * sizeof(char));
strcpy(registro_votos_global[total_votos_emitidos], id_usuario);
total_votos_emitidos++;

```

3.3 Mecanismo de Verificación de Fraude Electoral (Integridad Física)

El sistema cuenta con un sistema de verificación cruzada. Cuando el administrador desea visualizar los resultados, la aplicación lee ElectionInfo.txt y compara que los nombres y estados coincidan exactamente con la estructura de candidatos residente en memoria. Si un archivo ha sido modificado externamente de manera maliciosa, el software bloquea el escrutinio y emite una alerta crítica de fraude.

4. Pruebas, Validación y Demostración

El software fue compilado exitosamente para arquitecturas de 32 y 64 bits (votaciones.exe usando MinGW GCC 6.3.0). Las pruebas funcionales se dividen en tres escenarios principales:

4.1 Caso de Prueba 1: Inicialización del Proceso Electoral

1. El Administrador inicia sesión con el usuario POLIADMIN y clave admin123.
2. Selecciona la opción **1 (Nuevas Votaciones de AEIS)**.
3. Configura el año actual (2026) y el número inicial de candidatos (por ejemplo, 2 listas: *Lista A* y *Lista B*).
4. **Verificación:** El sistema escribe automáticamente los archivos de control ElectionInfo.txt, candidate1.txt hasta candidate4.txt (incluyendo votos blancos y nulos), y autogenera el padrón oficial padron.txt con 20 registros ordenados para permitir la búsqueda binaria.

4.2 Caso de Prueba 2: Panel de Votación del Estudiante

1. Se accede al **Panel de Estudiantes**.
2. Se ingresa el código 202311527 (válido dentro de padron.txt).
3. Se selecciona la opción de voto.
4. **Verificación:** El sistema suma el voto al candidato correspondiente, guarda en memoria dinámica el ID para evitar que vote de nuevo, y actualiza de inmediato el archivo .txt del candidato. Si el mismo usuario intenta votar de nuevo, el sistema le niega el acceso desplegando el mensaje: "El codigo ingresado YA REGISTRA UN VOTO."

4.3 Caso de Prueba 3: Cierre y Exportación de Resultados

1. Una vez alcanzado el cupo máximo de votaciones o forzado desde el panel de administrador, se selecciona la opción **3 (Ver Resultados y Exportar Acta)**.
2. El sistema recupera los datos y aplica la ordenación por selección descendente.
3. **Verificación:** Imprime el cuadro detallado con los resultados finales en formato tabular y exporta el archivo Acta_Electoral_AEIS.txt detallando:
 - Porcentaje de Participación Efectiva y Ausentismo.
 - Ranking electoral estructurado.
 - Dignidad Ganadora electa, omitiendo los votos en blanco y nulos para la proclamación formal.

1. Conclusiones del Proyecto

Optimización del tiempo de validación de identidad

La integración del algoritmo de **Búsqueda Binaria** sobre el arreglo unidimensional del padrón electoral (ordenado previamente de manera lexicográfica) demostró una reducción drástica en los tiempos de verificación de accesos, operando bajo una complejidad temporal. Esto mitiga la formación de cuellos de botella en las mesas de votación presenciales de la FIS y elimina el riesgo de colisiones de datos en comparación con las búsquedas secuenciales tradicionales de orden.

Garantía de integridad y prevención de fraude

El diseño e implementación del mecanismo de **verificación cruzada e integridad física** probó ser totalmente efectivo para detectar manipulaciones externas en caliente sobre la persistencia en disco duro (ElectionInfo.txt y ficheros independientes de candidatos). Al contrastar la información en tiempo real, el software mitiga vulnerabilidades de inyección o alteración física de datos, bloqueando el escrutinio si la simetría de la información se ve comprometida y asegurando la legitimidad de las dignidades electas.

Flexibilidad y escalabilidad de las listas

El uso correcto de la **gestión de memoria dinámica** a través de las primitivas de asignación (malloc y realloc) en la función de inserción dinámica resolvió la problemática de escalabilidad del sistema. Se logró la inserción estructurada de listas adicionales en tiempo de ejecución sin alterar el código fuente ni requerir una recompilación, manteniendo siempre el control lógico y el desplazamiento correcto de las opciones obligatorias correspondientes al Voto Blanco y Voto Nulo al final de la matriz.

Transparencia y auditoría automatizada

La aplicación del paradigma de **recursividad** para el conteo acumulativo de votos finales, en conjunto con el formateo modularizado de reportes, garantiza

un proceso de escrutinio auditable, transparente e inmediato. La generación automática en tiempo real del archivo Acta_Electoral_AEIS.txt provee al Tribunal Electoral de la FIS métricas instantáneas y exactas sobre el ausentismo y el porcentaje de participación efectiva, eliminando el error humano en la fase de totalización